

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

04/07/2024

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

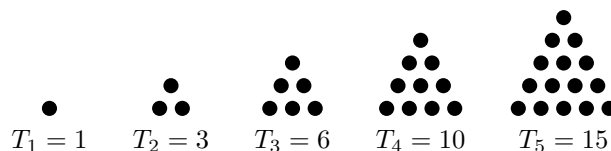
Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Triangular numbers are integers that count objects arranged in a triangle. The value of the n th triangular number is defined by the following equation:

$$T_n = \frac{n(n+1)}{2}$$

The zero-th triangular number is defined equal to zero ($T_0 = 0$) and there are no preceding triangular numbers (in other words, n cannot be less than zero). Examples of triangular numbers are shown in the following picture.



Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with the ability to compute the n th triangular number using a new **expression operator**, called **tri**, with the following syntax:

tri ($\langle exp. \rangle$)

The only **argument** to the `tri` operator is an **arbitrary expression** which specifies the index of the triangular number to compute. For example, `tri(5)` will compute the value 15. The operator is also **applicable to negative numbers**, but it always returns **zero**. For example, `tri(-10)` has the value 0. In your solution, it is **not required** to perform **constant folding**.

Example

The following program shows an example of use of the `tri` operator.

```
int i = 0;
// Print T_1, T_2, ... T_10
while (i < 10) {
    write(tri(i + 1));
    i = i + 1;
}
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (3 points)
2. Define the syntactic rules or the modifications required to the existing ones. (4 points)
3. Define the semantic actions needed to implement the required functionality. (18 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..5579b79 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,7 @@ ID      [a-zA-Z_][a-zA-Z0-9_]*
"return"      { return RETURN; }
"read"        { return READ; }
"write"       { return WRITE; }
+"tri"        { return TRI; }

{ID}          { yyval.svalue=strdup(yytext); return IDENTIFIER; }
{DIGIT}+      { yyval.intval = atoi( yytext ); }
diff --git a/acse/Acse.y b/acse/Acse.y
index 5e31c71..8d2154f 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -125,6 +125,7 @@ extern void yyerror(const char*);
%token RETURN
%token READ
%token WRITE
+%token TRI

%token <label> DO
%token <while_stmt> WHILE
@@ -544,6 +545,30 @@ exp: NUMBER    { $$ = create_expression ($1, IMMEDIATE); }
                                (program, exp_r0, $2, SUB);
                                }
                                }
+
+ | TRI LPAR exp RPAR
+ {
+     // Ensure the expression value is loaded into a register
+     int r_n;
+     if ($3.expression_type == IMMEDIATE)
+         r_n = gen_load_immediate(program, $3.value);
+     else
+         r_n = $3.value;
+     // Reserve a register for the result, and generate an instruction to
+     // initialize it to zero.
+     int r_res = gen_load_immediate(program, 0);
+     $$ = create_expression(r_res, REGISTER);
+     // Generate a sub+branch sequence that skips over the computation of
+     // the triangular number if the expression is negative
+     gen_subi_instruction(program, REG_0, r_n, 0);
+     t_axe_label *l_zero = newLabel(program);
+     gen_blt_instruction(program, l_zero, 0);
+     // Generate the computation of the triangular number
+     gen_addi_instruction(program, r_res, r_n, 1);
+     gen_mul_instruction(program, r_res, r_res, r_n, CG_DIRECT_ALL);
+     gen_divi_instruction(program, r_res, r_res, 2);
+     // Assign the label to the end of the code
+     assignLabel(program, l_zero);
+ }
+
;

%%
```

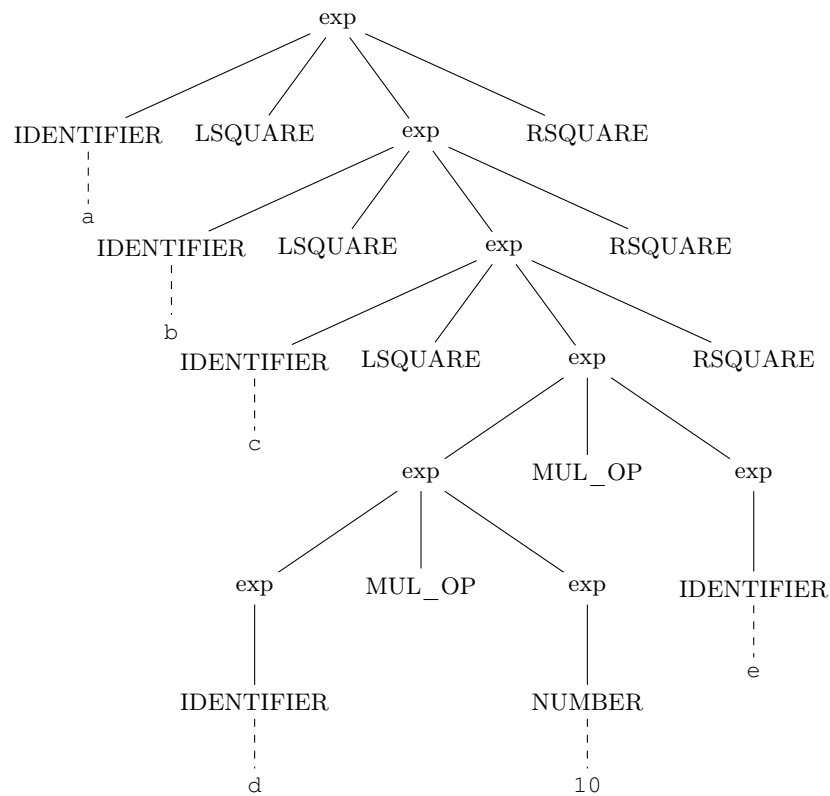
```
diff --git a/tests/tri/tri.src b/tests/tri/tri.src
new file mode 100644
index 0000000..ad672a2
--- /dev/null
+++ b/tests/tri/tri.src
@@ -0,0 +1,6 @@
+int i = 0;
+
+while (i < 15) {
+  write(tri(i - 5));
+  i = i + 1;
+}
```

4. Given the following Lance code snippet:

```
a[b[c[d*10*e]]]
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the exp nonterminal*. (5 points)

Solution



5. (**Bonus**) Briefly discuss in plain English words why it is **not necessary** to provide precedence and associativity declarations for the `tri` operator.

Additionally, show an alternative syntax for `tri` that **does require** the declaration of its precedence and associativity with respect to other expression operators.

Solution

The syntax of the operator `tri` as required by the text allows for an unambiguous grammar without defining precedence and associativity rules; hence they are not necessary. Specifically, the mandatory pair of parentheses around the expression argument forces it to have a single derivation tree.

On the other hand, if the operator did *not* require the parentheses around the expression, then it would have been necessary to define the precedence rules for `tri` to enforce the non-ambiguity of the grammar. For example, without precedence rules, the string

tri 1 + 2

allows both of the following parse trees:

